

Note:

This page explains the complete system, but you will start from phase 1.

Create a Bank system app which contains 3 modules:

1. Client Module

- a. Each client contains: int id, string name, string password, double balance.
- b. Clients can login to the system using id and password.
- c. Clients can deposit an amount of money.
- d. Clients can withdraw an amount of money.
- e. Clients can check their balance.
- f. Clients can transfer money to another client.

2. Employee Module

- a. Each employee contains: string name, int id, string password, double salary.
- b. Employees can login to the system using id and password.
- c. Employees can add new Clients.
- d. Employees can search for clients by id.
- e. Employees can list all clients.
- f. Employees can edit info of clients.
- g. Employees can display their info.

3. Admin Module

- a. Admin will be the same as Employee.
- b. Admins can add new Employees.
- c. Admins can search for Employees by id.
- d. Admins can list all Employees.
- e. Admins can edit info of Employees.
- f. Admins can display their info.

Hint:

You can create a Person Class that contains name, id, and password, then let the Client and Employee inherit all data from Person. Admin could inherit from Employee.

Phase 1

Create the following classes:

1. Client class, which contains the following:

- a. Int id, string name, string password, double balance.
- b. Setter functions
 - i. setName: the name must be alphabetic chars and min size 3 and max size 20.
 - ii. setPassword: Password must be with min size 8 and max size 20 without spaces.
 - iii. Min balance is 1500.
- c. Getter functions.
- d. void deposit (double amount).
- e. void withdraw (double amount).
- f. void transferTo (double amount, Client& recipient).
- g. void checkBalance ().
- h. Display function.

2. Employee, Class which contains the following:

- a. Int id, string name, string password, double salary.
- b. Setter functions
 - i. setName: the name must be alphabetic chars and min size 3 and max size 20.
 - ii. setPassword: Password must be with min size 8 and max size 20 without spaces.
 - iii. Min Salary 5000.
- c. Getter functions.
- d. Display function.

3. Admin Class, which contains the following:

- a. Int id, string name, string password, double salary.
- b. Setter functions
 - i. setName: the name must be alphabetic chars and min size 3 and max size 20.
 - ii. setPassword: Password must be with min size 8 and max size 20 without spaces.
 - iii. Min Salary 5000.
- c. Getter functions.
- d. Display function.

Hint:

You can create a Validation class (Utility Class) that contains all validations that you will need as static methods and use these validations in the rest of other classes.

End of Phase 1

Phase 2

1. Create the following text files.
 - a. Clients.txt to save client info.
 - b. Employee.txt to save employee info.
 - c. Admin.txt to save admin info.
2. Create DataSourceInterface as an abstract class contains the following:
 - a. Abstract void addClient(Client obj).
 - b. Abstract void addEmployee(Employee obj).
 - c. Abstract void addAdmin(Admin obj).
 - d. Abstract void getAllClients().
 - e. Abstract void getAllEmployees().
 - f. Abstract void getAllAdmins().
 - g. Abstract void removeAllClients().
 - h. Abstract void removeAllEmployees().
 - i. Abstract void removeAllAdmins().
3. Create Parser class to read string line and split this string into (id, name, password, and balance or salary) contains:
 - a. static vector<string> split(string line).
 - b. static Client parseToClient(string line).
 - c. static Employee parseToEmployee(string line).
 - d. static Admin parseToAdmin(string line).
4. Create FilesHelper class that saves to and gets from txt files contains:
 - a. static void saveLast(string fileName, int id)
 - b. static int getLast(string fileName).
 - c. static void saveClient(Client c).
 - d. static void saveEmployee(string fileName, string lastIdFile, Employee e).

- e. static void getClients().
- f. static void getEmployees().
- g. static void getAdmins().
- h. static void clearFile(string fileName, string lastIdFile).

5. Create FileManager Class to implement the DataSourceInterface class:

- a. addClient should save client info in clients.txt.
- b. addEmployee should save employee info in employees.txt.
- c. addAdmin should save employee info in admins.txt.
- d. getAllClients(), getAllEmployees(), and getAllAdmins() should retrieve data from files.
- e. removeAllClients(), removeAllEmployees(), and removeAllAdmins() should remove all data from the files.

6. Add the following to the employee class:

- a. void addClient(Client& client).
- b. Client* searchClient(int id).
- c. void listClient().
- d. void editClient(int id, string name, string password, double balance).

7. Add the following to the admin class:

- a. void addClient(Client& client).
- b. Client* searchClient(int id).
- c. void listClient().
- d. void editClient(int id, string name, string password, double balance).
- e. void addEmployee(Employee& employee).
- f. Employee* searchEmployee(int id).
- g. void editEmployee(int id, string name, string password, double salary).
- h. void listEmployee().

End of Phase 2

Phase 3

1. Create **ClientManger** class that contains:
 - a. static void printClientMenu().
 - b. static void updatePassword(Person* person).
 - c. static Client* login(int id, string password).
 - d. static bool clientOptions(Client* client).
2. Create **EmployeeManager** class that contains:
 - a. static void printEmployeeMenu().
 - b. static void newClient(Employee* employee).
 - c. static void listAllClients(Employee* employee).
 - d. static void searchForClient(Employee* employee).
 - e. static void editClientInfo(Employee* employee).
 - f. static Employee* login(int id, string password).
 - g. static bool employeeOptions(Employee* employee).
3. Create **AdminManager** class that contains:
 - a. static void printEmployeeMenu().
 - b. static Admin* login(int id, string password).
 - c. static bool AdminOptions(Admin* admin).
4. Create **Screens** class that contains:
 - a. static void bankName().
 - b. static void welcome().
 - c. static void loginOptions().
 - d. static int loginAs().
 - e. static void invalid(int c).
 - f. static void logout().
 - g. static void loginScreen(int c).
 - h. static void runApp().

End of Phase 3